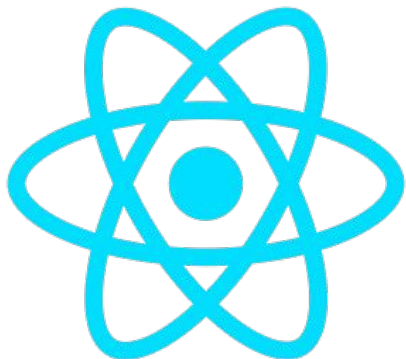




The Ultimate React Course



 @newtonschool

Lecture 17: State

-Priyanshu Verma



Managing Data in React

In JavaScript, **we use variables to store data**. Can we use them in React to update the UI? Let's find out!



Javascript Variables to store values



Can we use them
in React too??

Managing Data in React

Here we have built an app where we can increment the value of count by clicking on *Increment* button.

```

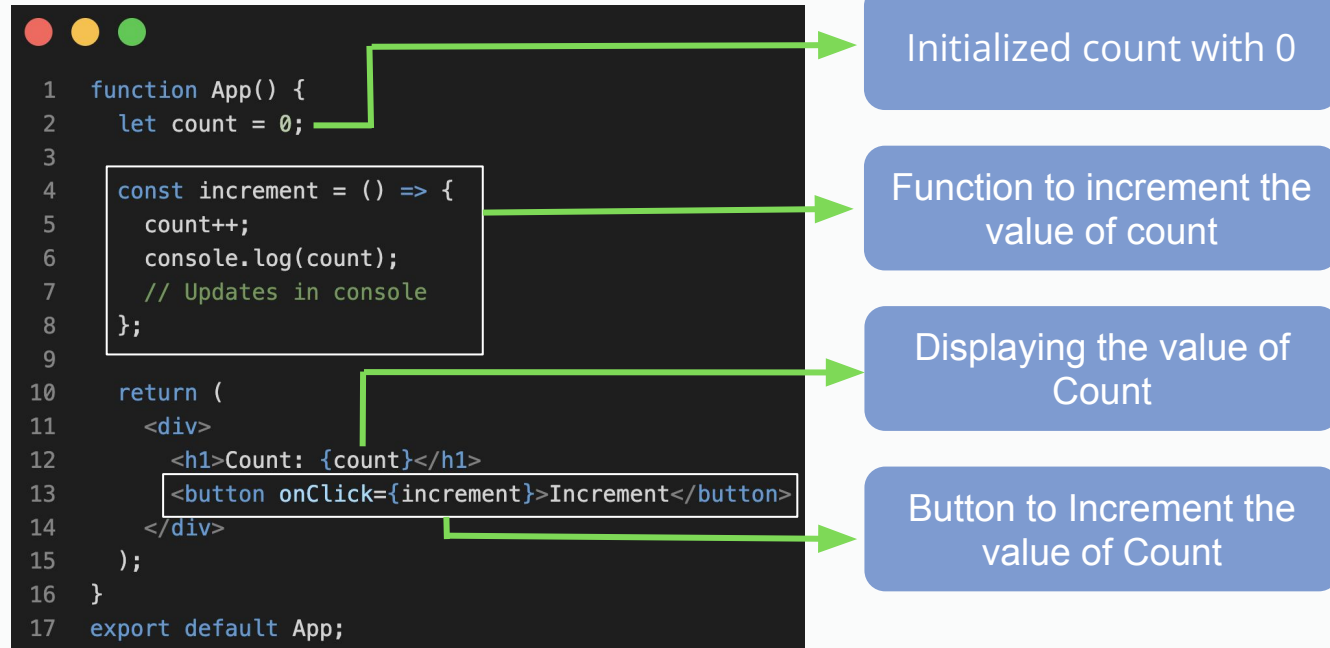
1  function App() {
2    let count = 0;
3
4    const increment = () => {
5      count++;
6      console.log(count);
7      // Updates in console
8    };
9
10   return (
11     <div>
12       <h1>Count: {count}</h1>
13       <button onClick={increment}>Increment</button>
14     </div>
15   );
16 }
17 export default App;

```



Managing Data in React

Here we have built an app where we can increment the value of count by clicking on *Increment* button.



```

1  function App() {
2    let count = 0;
3
4    const increment = () => {
5      count++;
6      console.log(count);
7      // Updates in console
8    };
9
10   return (
11     <div>
12       <h1>Count: {count}</h1>
13       <button onClick={increment}>Increment</button>
14     </div>
15   );
16 }
17 export default App;
  
```

- Initialized count with 0
- Function to increment the value of count
- Displaying the value of Count
- Button to Increment the value of Count

Managing Data in React

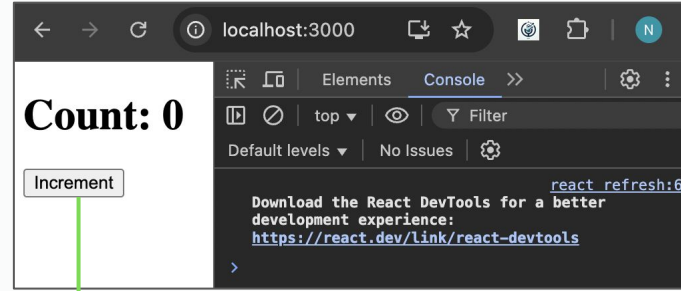
Let's understand how it works:-

Initial State

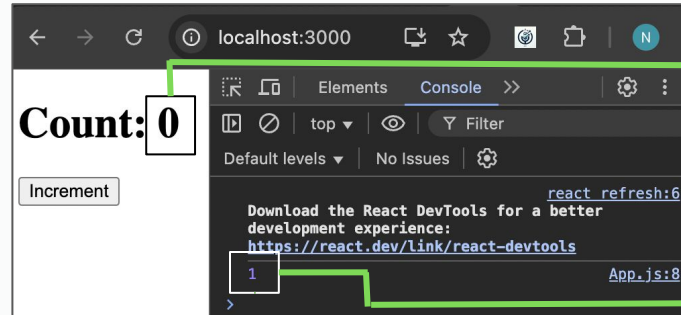
```

1  function App() {
2    let count = 0;
3
4    const increment = () => {
5      count++;
6      console.log(count);
7      // Updates in console
8    };
9
10   return (
11     <div>
12       <h1>Count: {count}</h1>
13       <button onClick={increment}>Increment</button>
14     </div>
15   );
16 }
17 export default App;

```



After first click in Increment



Not reflected
in UI

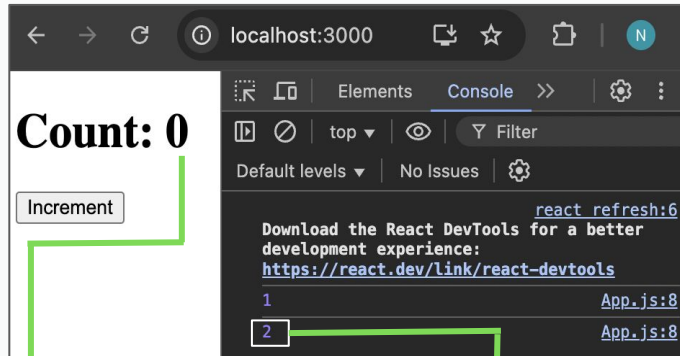
But

Value gets
updated in
console

Managing Data in React

Oh, let's try one more click:-

After second click in Increment



UI still remains
unchanged

But

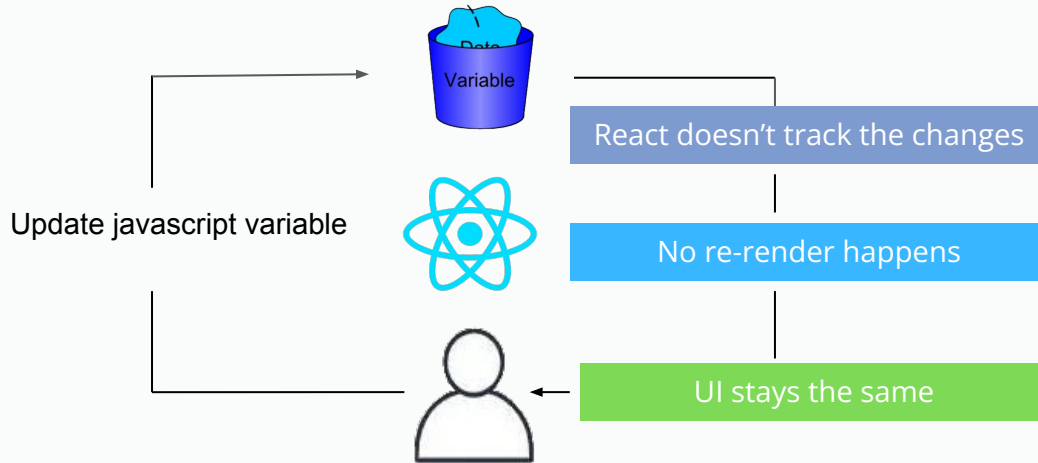
Value incremented
from 1 to 2

Holly, molly, why is
it not updating?



Why did it happen?

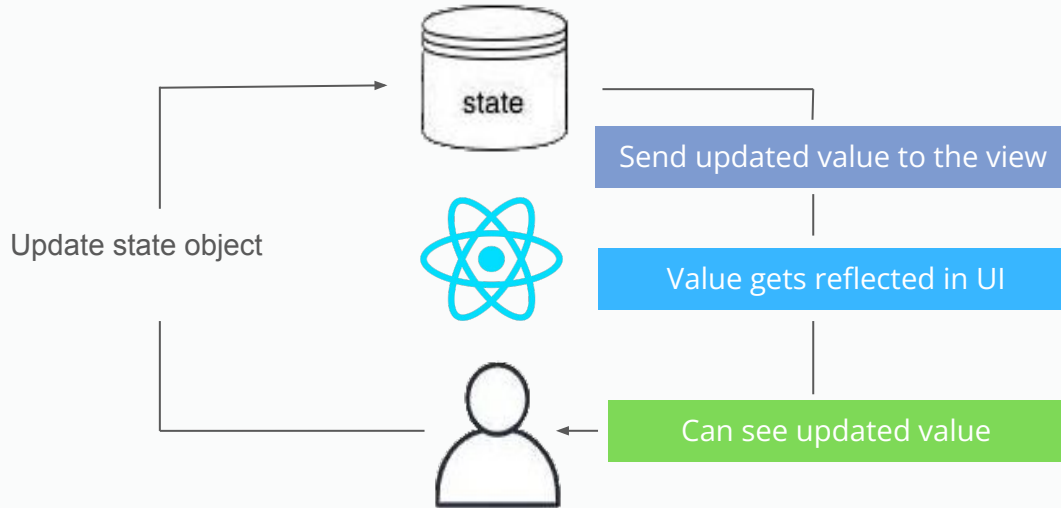
It happened because **React doesn't track normal variable changes**. React only re-renders when state or props change.



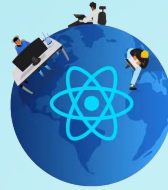
Even though we change the value UI doesn't changes and if we reload the page then value returns to its default.

Why did it happen?

React re-renders only when state or props change, ensuring the UI always reflects the latest state accurately. 🚀



Use state to re-render the UI when a value changes; avoid it for values that don't affect rendering.



Understanding React State

Introduction – What is State?

State is like a theater play 🎭—the stage (UI) must always reflect the latest scene.

Actors (state values) change their poses.



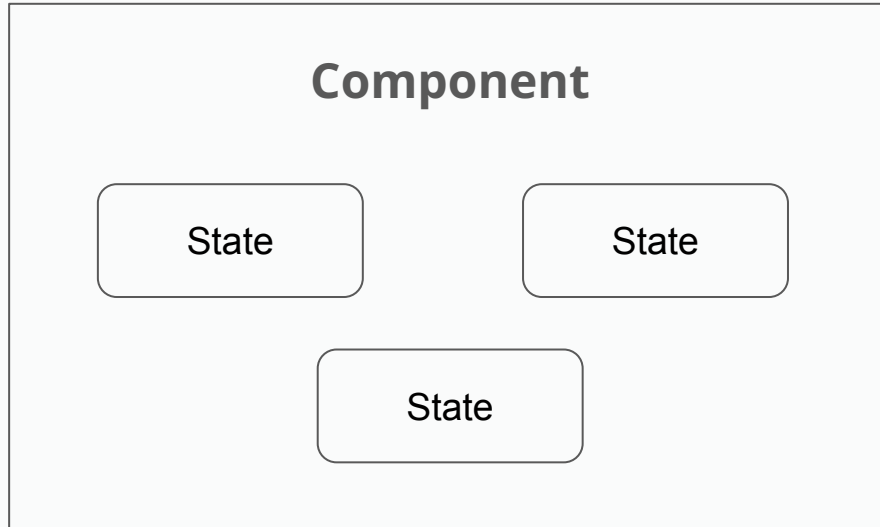
Actors in
different dance
poses(state
values)

The director (React)
ensures the
audience sees the
right scene by
updating the stage
when needed.

Stage (UI)

Introduction – What is State?

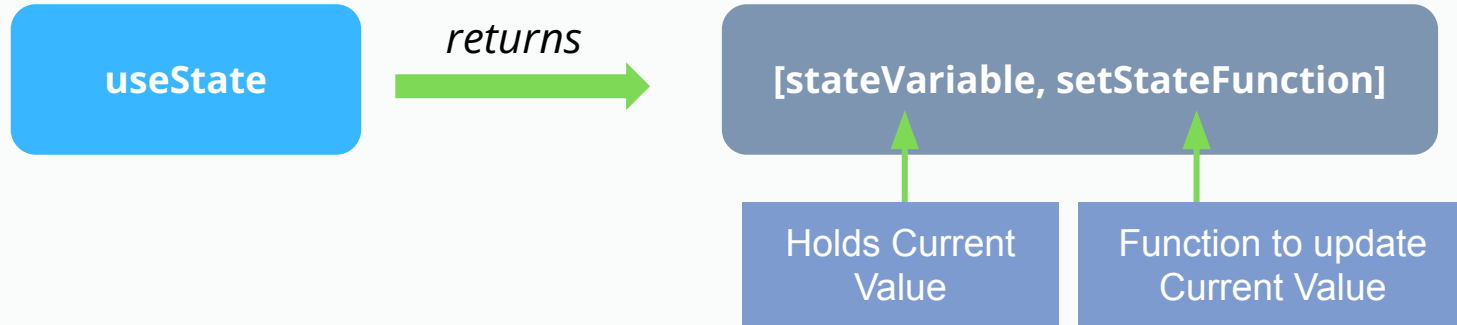
Different states in react gets stored in the memory. It stores values that change over time and updates the UI accordingly.



Just like each play has several actors having different poses, each component in React can maintain multiple states.

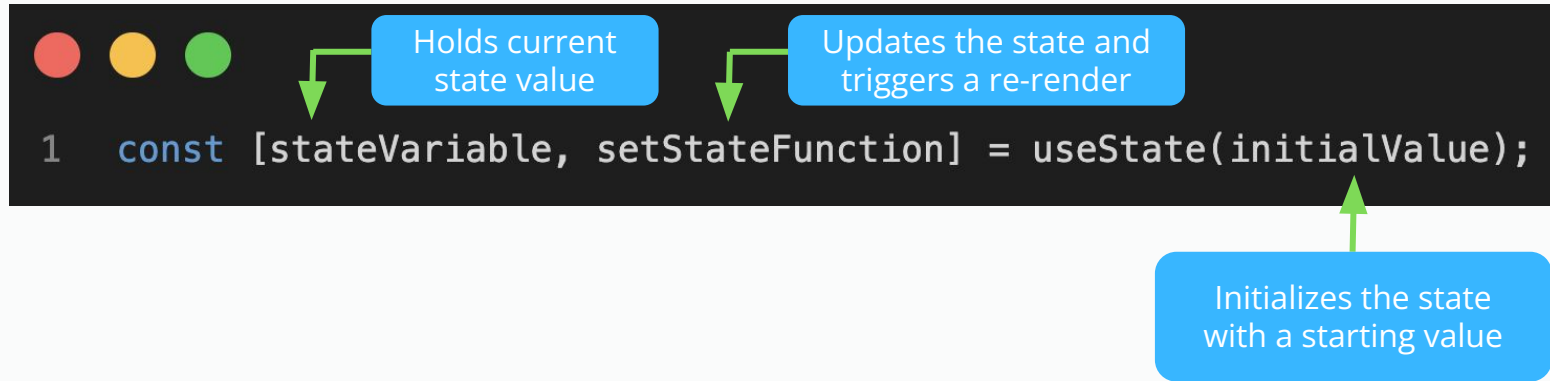
Understanding useState Hook

`useState` is a React Hook that lets us add state to functional components. It returns an array with two elements:



Declaring state with useState hook

We use the `useState` hook to **initialize a React state variable**, allowing components to track and update values dynamically.



useState hook usage

Let's create a counter where we can increment or decrement the counter by clicking buttons, for storing count variable, here we will use state variable.

```

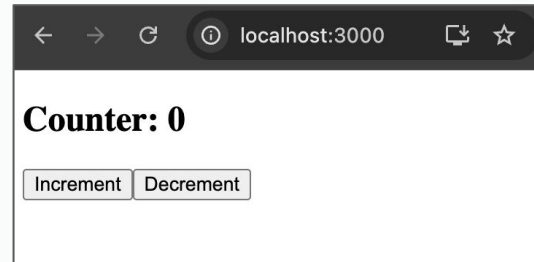
1  import { useState } from "react";
2
3  function App() {
4    // Initializing state with useState
5    const [count, setCount] = useState(0);
6
7    return (
8      <div>
9        <h2>Counter: {count}</h2>
10       <button>Increment</button>
11       <button>Decrement</button>
12     </div>
13   );
14 }
15
16 export default App;

```

At first we must import the useState hook from the React library

We initialized the state variable with 0

State variable



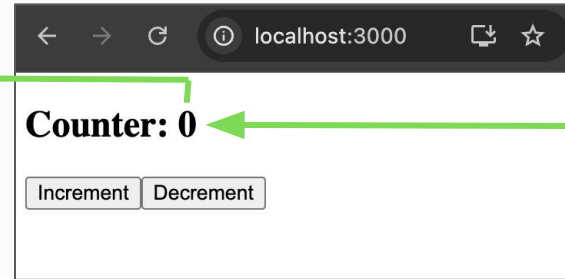
useState hook usage

Let's create a counter where we can increment or decrement the counter by clicking buttons, for storing count variable, here we will use state variable.

```

1  import { useState } from "react";
2
3  function App() {
4    // Initializing state with useState
5    const [count, setCount] = useState(0);
6
7    return (
8      <div>
9        <h2>Counter: {count}</h2>
10       <button>Increment</button>
11       <button>Decrement</button>
12     </div>
13   );
14 }
15
16 export default App;

```



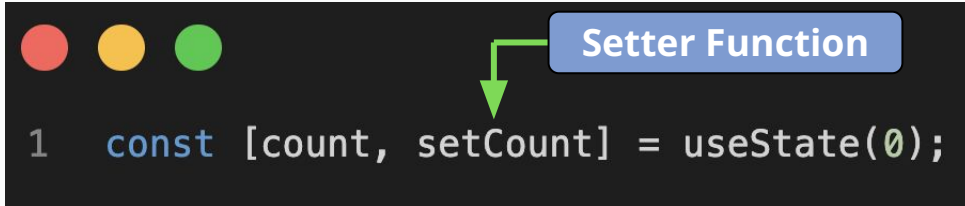
Now we need to add an event handler to handle events:-

1. User Clicks on Increment button
2. User Clicks on Decrement button

Accordingly
value in UI
should
change

useState: setter function

The **setter function** returned by `useState` is used to update the state value and trigger a re-render.



```
1  const [count, setCount] = useState(0);
```

```
setCount(count + 1);
```



Increments the value of count variable state

```
setCount(count - 1);
```



Decrements the value of count variable state

useState: setter function

Let's write complete code for Counter app using useState and its setter function:-

```

1  import { useState } from "react";
2
3  function App() {
4    // Initializing state with useState
5    const [count, setCount] = useState(0);
6
7    return (
8      <div>
9        <h2>Counter: {count}</h2>
10       <button onClick={() => setCount(count + 1)}>
11         Increment
12       </button>
13       <button onClick={() => setCount(count - 1)}>
14         Decrement
15       </button>
16     </div>
17   );
18 }
19
20 export default App;

```

Event Handlers

Incrementing and
decrementing using
setCount() setter function
when user clicks the
button.

Testing our app: Increment Button

Let's try clicking on the 'Increment' button:

Before Click:-

Counter: 0

2nd Click:-

Counter: 1

3rd Click:-

Counter: 2

Increment button is working as
expected

Testing our app: Decrement Button

Let's try clicking on the 'Decrement' button:

Before Click:-

Counter: 2

2nd Click:-

Counter: 1

3rd Click:-

Counter: 0

Decrement button too is working
as expected

Multiple State Variables — The Idea

In React, different pieces of data often serve different purposes.

Keeping state separate:

- Improves clarity
- Reduces bugs
- Makes updates easier

```
const [name, setName] = useState("");  
const [age, setAge] = useState(0);
```

Key Thought:

👉 One responsibility = one state

Multiple State Variables — How

In React, different pieces of data often serve different purposes.

Explanation:

- **name** and **age** are independent
- Each state controls its own updates
- Changing one does not affect the other

```
const [name, setName] = useState("");  
const [age, setAge] = useState(0);
```

Rule of Thumb:

- Simple & unrelated data → **separate states**

Single vs Multiple State

Should name and age be in one state or two?

Guideline:

- Simple data → separate states
- Closely related data → object state

Why this matters:

- Prevents unnecessary updates
- Keeps logic understandable

```
function Profile() {  
  const [name, setName] = useState("Bheem");  
  const [age, setAge] = useState(12);  
  
  return (  
    <div>  
      <p>Name: {name}</p>  
      <p>Age: {age}</p>  
  
      <button onClick={() => setAge(age + 1)}>  
        Increase Age  
      </button>  
    </div>  
  );  
}
```

Multiple State Variables

What's happening:

- **name** and **age** are **independent**
- Updating **age** does NOT touch **name**
- Easy to manage for simple values

```
function Profile() {  
  const [name, setName] = useState("Bheem");  
  const [age, setAge] = useState(12);  
  
  return (  
    <div>  
      <p>Name: {name}</p>  
      <p>Age: {age}</p>  
  
      <button onClick={() => setAge(age + 1)}>  
        Increase Age  
      </button>  
    </div>  
  );  
}
```

Single Object State(Combined State)

What's happening:

- **name** and **age** belong to the **same entity**
- Spread operator keeps old values safe
- Used in forms, profiles, grouped data

```
function Profile() {  
  const [user, setUser] = useState({  
    name: "Bheem",  
    age: 12  
  });  
  
  return (  
    <div>  
      <p>Name: {user.name}</p>  
      <p>Age: {user.age}</p>  
  
      <button onClick={() =>  
        setUser({ ...user, age: user.age + 1 })  
      }>  
        Increase Age  
      </button>  
    </div>  
  );  
}
```

Re-render on State Change

```
const Component = () => {  
  const [state, setState] = useState();  
  
  return ...  
};
```

2: re-renders

1: state change

In React:

- When state changes
- The **component function runs again**
- JSX is recalculated
- DOM updates efficiently

Understanding Re-render

Explanation:

- Page loads → component runs → log appears
- Button clicks → state changes
- State change → component runs again
- **console.log** prints again

```
import { useState } from "react";

function Demo() {
  const [count, setCount] = useState(0);

  // Runs on every render (initial + re-render)
  console.log("Component rendered");

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}

export default Demo;
```

Object State — When and Why

Why object state is needed:

- Forms
- User profiles
- Grouped data

```
const [user, setUser] = useState({  
  name: "",  
  age: ""  
});
```

Concept:

- One state holds **related properties**
- Easier to manage as a group

Object State — When and Why

What is Object State?

- Used when multiple related values belong together
- Helps keep data organized and meaningful
- Common in forms, user profiles, settings

One state = one real-world entity

```
const [user, setUser] = useState({  
  name: "Bheem",  
  age: 12  
});
```

Updating Object State (Correct Way)

Why Spread (...user)?

- Copies existing properties
- Prevents accidental data loss
- Ensures safe updates

```
const [user, setUser] = useState({  
  name: "Bheem",  
  age: 12  
});
```

```
setUser({  
  ...user,    // keep existing data  
  age: 13     // update only age  
});  
  
console.log(user);
```

Common Mistakes:

! What Went Wrong?

- Old **name** is **lost**
- Entire object is replaced
- React does **not merge objects automatically**

```
setUser({  
  age: 13  
});  
  
console.log(user);
```

```
setUser({  
  ...user,  
  age: 13  
});
```

Objects are replaced, not merged - use spread operator

Array State: Why & Use Cases

What is Array State?

- Used when your data is a **list of items**
- Very common in real applications

Where do we use it?

- Todo lists
- Product carts
- Student lists
- Notifications

```
const [items, setItems] = useState([]);
```

`items` → current list

`setItems` → updates the list

Updating Array State

Add an item

- Creates a **new array**
- Keeps old items + adds new one

```
const [items, setItems] = useState([]);
```

```
setItems(items.filter(item => item !== "Pen"));
```

```
setItems([...items, "Pen"]);
```

Remove an item

- Filter returns a **new array**
- Original array stays untouched

Golden Rule (Highlight):

- Never modify array directly (**push, pop**)
- Always create a **new array**

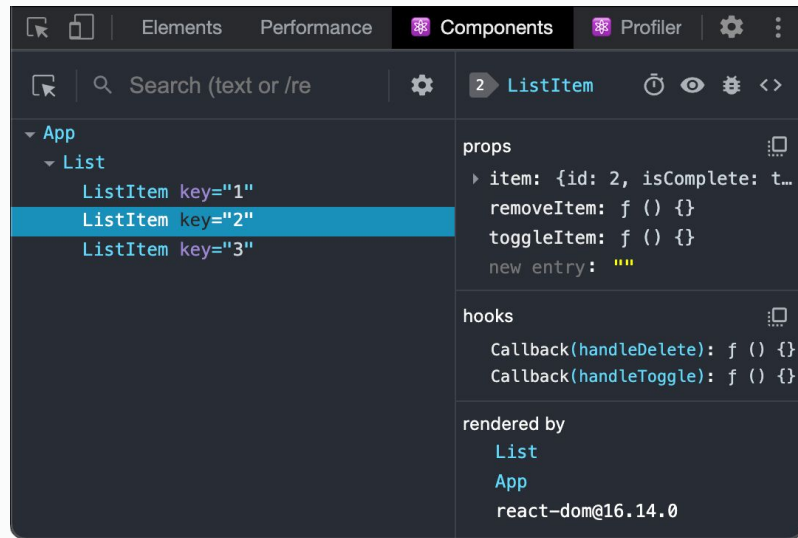
React DevTools — Why It Matters

What React DevTools show:

- Components tree
- Live state values
- Prop updates

Why developers use it:

- Debugging
- Understanding app behavior
- Industry-standard workflow



Final Reminder:

👉 React state is invisible without proper tools

References

1. **MDN React Guide:** Comprehensive and beginner-friendly documentation for React
[https://developer.mozilla.org/en-US/docs/Learn/Tools and testing/Client-side JavaScript frameworks/React getting started](https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started)
2. **Websites and Tutorials:**
 - a.  w3schools: <https://www.w3schools.com/REACT/DEFAULT.ASP>
 - b.  codecademy: <https://www.codecademy.com/learn/react-101>
3. **Other Useful Resources**
 - a.  React GitHub Repository: <https://github.com/facebook/react>
 - b.  React Patterns & Best Practices: <https://reactpatterns.com/>

**Thanks
for
watching!**